15/oct/2026

3) Human Errors

- Humans are known to be unreliable
- How do we make our systems reliable, inspite of unreliable humans?

a) Design systems in a way that minimizes opportunities for errors.
for examples well-designed abstractions, APIs, and admin interfaces makes it easy to do "the right thing" and discourage "the wrong thing".

b) Decouple the places where people make the most mistakes from the places where they can cause failures. In particular, provide fully featured non-production sandbox environments where people can explore and experiment safely, using real data, without affecting real users.

c) Test throughly at all levels, from unit tests to whole system integration and manual tests

d) Allow quick and easy recovery from human errors, to minimise the impact in case of failures.

e.g. Make it fast to rollback configuration changes, roll out new code gradually and provide tools to recompute data

e) Setup ~~of~~ detailed and clear monitoring, such as performance metric and error rates. In other engineering disciplines this is referred to as telemetry.

Monitoring can show us early warning signals and allow us to check whether any assumptions or constraints are being violated. When a problem occurs, metrics can be invaluable in diagnosing the issue

f) Implement good management practices and training

Scalability

NOTES It is a term we use to describe a system's ability to cope with increased load.

Note, however, that is not a one dimensional label that we can attach to a system:

It is meaningless to say "X is scalable"



or "Y doesn't scale". Rather, discussing scalability means considering questions like "If the system grows in particular way, what are our options for coping with the growth?" and how can we add computing resources to handle the additional load?"

Describing Load

Load can be described with a few numbers which we call load parameters.

The best choice of parameters depends on the architecture of your system:

a) It may be requests per second to a web server.
b) The ratio of reads to write in a database
c) the number of simultaneously active users in a chat room,
d) The hit rate on a cache etc



IMPORTANT

NOTES

FEBRUARY '13

M	T	W	T	F	S	S
				1	2	3
4	5	6	7	8	9	10
11	12	13	14	15	16	17
18	19	20	21	22	23	24
25	26	27	28			

Describing Performance

- You can look it into two ways:

1) When you increase the load and keep the system resources (CPU, Memory, network bandwidth etc.) unchanged, how is the performance of your system affected?

2) When you increase a load parameter, how much do you need to increase the resources if you want to keep the performance unchanged

Both questions require performance numbers, so let's look briefly at describing the performance of a system.

In a batch processing system such as Hadoop, we usually care about throughput — the number of records we can process per second or the total time it takes to run a job on a dataset of a certain size.

In online systems, service response time is critical — i.e. the time between a client sending a request and receiving a response.

060-305 Wk 09

Latency and Response time

Latency and response time are often used synonymously, but they are not the same.

- The response time is what the client sees: besides the actual time to process the request it involves network delays and queuing delays

- Latency is the duration that a request is waiting to be handled - during which it is latent, awaiting service.

NOTES